FRI 프로토콜 (Fast Reed-Solomon IOP)

목차

1. FRI 개요
2. Reed-Solomon 코드와 Low Degree Testing
3. FRI Commit Phase
4. FRI Query Phase
5. FRI Verification

FRI 개요

FRI (Fast Reed-Solomon Interactive Oracle Proof of Proximity)는 plonky2의 핵심 프로토콜로, 주어진 함수가 낮은 차수의 다항식인지 효율적으로 검증하는 방법입니다.

FRI의 목적

FRI는 다음과 같은 문제를 해결합니다:

- **문제**: Prover가 제공한 함수 f가 정말로 차수 d 이하의 다항식인가?
- **해결**: 전체 다항식을 보내지 않고도 낮은 차수임을 증명

FRI가 필요한 이유

PLONK 프로토콜에서:

1. Prover는 여러 다항식을 커밋합니다 (wires, permutation, quotient 등)

2. 이 다항식들이 올바른 차수를 가지는지 검증해야 합니다

3. FRI는 이를 효율적으로 검증하는 도구입니다

FRI의 핵심 아이디어

```
고차 다항식 P(x) → 저차 다항식 P'(x) → ... → 상수 다항식
```

- **Folding**: 다항식을 반복적으로 접어서 차수를 줄입니다
- **Random Challenge**: 각 단계에서 랜덤 챌린지 β를 사용하여 보안성 확보
- **Query**: 최종적으로 몇 개의 점만 확인하여 전체 다항식 검증

Reed-Solomon 코드와 Low Degree Testing

Reed-Solomon 코드

Reed-Solomon 코드는 다항식을 여러 점에서 평가한 값들의 집합입니다.

```
다항식 P(x) = a₀ + a₁ x + a₂ x² + ... + aₐxᵈ

Reed-Solomon 코드 = [P(ω⁰), P(ω¹), P(ω²), ..., P(ωⁿ⁻¹)]
```

여기서:

- d: 다항식의 차수
- n: 평가 점의 개수 (n >> d, 보통 n = d × rate)
- ω: n차 단위근 (primitive root of unity)

Low Degree Testing (LDT)

목표: 주어진 함수가 낮은 차수의 다항식에 가까운지 테스트

방법:

1. 함수를 여러 점에서 샘플링
2. 샘플링한 값들이 낮은 차수 다항식과 일치하는지 확인
3. 충분히 많은 점을 확인하면 높은 확률로 검증 가능

FRI의 장점:

- 전통적인 LDT: O(n) 복잡도
- FRI: O(log n) 복잡도 (훨씬 효율적!)

FRI Commit Phase

Commit Phase는 FRI의 첫 번째 단계로, 다항식을 반복적으로 접어서(fold) 차수를 줄이는 과정입니다.

fri_committed_trees 함수

```
fn fri_committed_trees<F: RichField + Extendable<D>, C: GenericConfig<D, F = F>
mut coeffs: PolynomialCoeffs<F::Extension>,
mut values: PolynomialValues<F::Extension>,
challenger: &mut Challenger<F, C::Hasher>,
fri_params: &FriParams,
final_poly_coeff_len: Option<usize>,
max_num_query_steps: Option<usize>,
```

```
) -> FriCommitedTrees<F, C, D>
```

입력:

- `coeffs` : LDE 다항식의 계수 (Low Degree Extension)
- `values` : 큰 도메인에서의 다항식 평가값
- `challenger` : Fiat-Shamir 변환을 위한 챌린저
- `fri_params` : FRI 파라미터 (reduction arity, cap height 등)

출력:

- Merkle Tree들의 벡터
- 최종 다항식의 계수

Folding 과정 상세 설명

1단계: 초기 설정

```
let mut trees = Vec::with_capacity(fri_params.reduction_arity_bits.len());
let mut shift = F::MULTIPLICATIVE_GROUP_GENERATOR;
```

- `trees` : 각 reduction 단계에서 생성되는 Merkle Tree를 저장
- `shift` : 코셋(coset) 이동을 위한 생성자

2단계: Reduction 루프

```
for arity_bits in &fri_params.reduction_arity_bits {
let arity = 1 << arity_bits;
// 1. 비트 역순 정렬
```

```
reverse_index_bits_in_place(&mut values.values);
// 2. 값들을 arity 크기로 묶기
let chunked_values = values
.values
.par_chunks(arity)
.map(|chunk: &[F::Extension]| flatten(chunk))
.collect();
// 3. Merkle Tree 생성
let tree = MerkleTree::<F, C::Hasher>::new(
chunked_values,
fri_params.config.cap_height
);
// 4. Merkle Cap 관찰 (Fiat-Shamir)
challenger.observe_cap(&tree.cap);
trees.push(tree);
// 5. Beta 챌린지 생성
let beta = challenger.get_extension_challenge::<D>();
// 6. 다항식 Folding
coeffs = PolynomialCoeffs::new(
coeffs
.coeffs
.par_chunks_exact(arity)
.map(|chunk| reduce_with_powers(chunk, beta))
```

```
.collect::<Vec<_>>(),
);
// 7. 다음 라운드 준비
shift = shift.exp_u64(arity  as u64);
values = coeffs.coset_fft(shift.into())
}
```

Folding의 수학적 원리

다항식 P(x)를 arity r로 접는 과정:

$$P(x) = P_0(x^r) + x \cdot P_1(x^r) + x^2 \cdot P_2(x^r) + \ldots + x^{r-1} \cdot P_{r-1}(x^r)$$

Beta 챌린지 β를 사용하여 새로운 다항식 P'(x) 생성:

$$P'(x) = P_0(x) + \beta \cdot P_1(x) + \beta^2 \cdot P_2(x) + \ldots + \beta^{r-1} \cdot P_{r-1}(x)$$

효과:

- 원래 다항식의 차수: d
- 접은 후 다항식의 차수: d/r
- 여러 번 반복하면 차수가 급격히 감소

예시: arity = 2인 경우

$$P(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + a_4 x^4 + a_5 x^5$$

```
분해:

P_even(x²) = a₀ + a₂ x² + a₄ x⁴

P_odd(x²) = a₁  + a₃ x² + a₅x⁴



P(x) = P_even(x²) + x·P_odd(x²)



Beta 챌린지 β 사용:

P'(x) = P_even(x) + β·P_odd(x)

= (a₀ + βa₁ ) + (a₂  + βa₃ )x + (a₄  + βa₅)x²



차수: 5 → 2 (절반으로 감소)
```

Beta 챌린지의 역할

보안성 확보:

- Beta는 Prover가 커밋한 후에 생성됩니다 (Fiat-Shamir)
- Prover는 Beta를 미리 알 수 없으므로 속일 수 없습니다
- 각 reduction 단계마다 새로운 Beta 사용

검증 가능성:

- Verifier는 같은 Beta를 사용하여 folding을 재현할 수 있습니다
- 원래 값과 접힌 값 사이의 일관성을 확인할 수 있습니다

Merkle Tree 구성

각 reduction 단계에서:

```
let  tree = MerkleTree::<F, C::Hasher>::new(
chunked_values,
fri_params.config.cap_height
);
```

구조:

```
Root (Cap)
/ \
... ...
/ \ / \
Leaf Leaf Leaf Leaf
[v₀] [v₁ ] [v₂ ] [v₃ ]
```

- **Leaf**: arity개의 평가값을 flatten한 데이터
- **Cap**: 상위 일부 노드들 (빠른 검증을 위해)
- **Commitment**: Cap을 해시하여 챌린저에 전달

최종 다항식 처리

```
// 고차 계수 제거 (항상 0이어야 함)
coeffs
.coeffs
.truncate(coeffs.len() >> fri_params.config.rate_bits);
```

```
// 최종 다항식 관찰
challenger.observe_extension_elements(&coeffs.coeffs);
```

이유:

- LDE (Low Degree Extension)에서는 고차 계수가 0입니다
- 실제 의미 있는 계수만 남깁니다
- 최종 다항식은 충분히 작아서 직접 전송 가능합니다

Commit Phase 요약

1. **입력**: 큰 차수의 다항식 P(x)
2. **반복**:
- 평가값을 arity개씩 묶어 Merkle Tree 생성
- Merkle Cap을 챌린저에 전달
- Beta 챌린지 받기
- 다항식을 Beta로 접기 (차수 감소)
3. **출력**:
- 각 단계의 Merkle Tree들
- 충분히 작은 최종 다항식

FRI Query Phase

Query Phase는 Prover가 특정 위치의 값들과 Merkle Proof를 제공하는 단계입니다.

fri_prover_query_rounds 함수

```
fn  fri_prover_query_rounds<
F: RichField + Extendable<D>,
C: GenericConfig<D, F = F>,
const D: usize,
>(
initial_merkle_trees: &[&MerkleTree<F, C::Hasher>],
trees: &[MerkleTree<F, C::Hasher>],
challenger: &mut Challenger<F, C::Hasher>,
n: usize,
fri_params: &FriParams,
) -> Vec<FriQueryRound<F, C::Hasher, D>>
```

입력:

- `initial_merkle_trees` : 원본 다항식들의 Merkle Tree (wires, zs 등)
- `trees` : Commit Phase에서 생성된 FRI Merkle Tree들
- `challenger` : 쿼리 인덱스 생성을 위한 챌린저
- `n` : LDE 도메인 크기

출력:

- 각 쿼리 라운드의 증명들

Query 선택 메커니즘

```
challenger
.get_n_challenges(fri_params.config.num_query_rounds)
.into_par_iter()
.map(|rand| {
let  x_index = rand.to_canonical_u64() as usize % n;
fri_prover_query_round::<F, C, D>(
initial_merkle_trees,
trees,
x_index,
fri_params
)
})
.collect()
```

과정:

1. Challenger로부터 랜덤 챌린지 생성
2. 챌린지를 인덱스로 변환 (0 ~ n-1 범위)
3. 각 인덱스에 대해 쿼리 라운드 실행
4. 병렬 처리로 성능 최적화

보안성:

- 쿼리 인덱스는 Prover가 커밋한 후에 생성됩니다
- Fiat-Shamir 변환으로 예측 불가능합니다
- 여러 번 쿼리하여 soundness 확보

fri_prover_query_round 함수

```
fn fri_prover_query_round<
F: RichField + Extendable<D>,
C: GenericConfig<D, F = F>,
const D: usize,
>(
initial_merkle_trees: &[&MerkleTree<F, C::Hasher>],
trees: &[MerkleTree<F, C::Hasher>],
mut x_index: usize,
fri_params: &FriParams,
) -> FriQueryRound<F, C::Hasher, D>
```

단일 쿼리 라운드 처리:

1단계: Initial Trees Proof

```
let initial_proof = initial_merkle_trees
.iter()
.map(|t| (t.get(x_index).to_vec(), t.prove(x_index)))
.collect::<Vec<_>>();
```

- 원본 다항식들 (wires, zs 등)의 x_index 위치 값 가져오기
- 각 값에 대한 Merkle Proof 생성
- 이는 PLONK 프로토콜의 opening과 연결됩니다

2단계: FRI Reduction Steps

```
for (i, tree) in  trees.iter().enumerate() {
let  arity_bits = fri_params.reduction_arity_bits[i];
// 코셋 내 인덱스 계산
let  coset_index = x_index >> arity_bits;
// 해당 코셋의 모든 평가값 가져오기
let  evals = unflatten(tree.get(coset_index));
// Merkle Proof 생성
let  merkle_proof = tree.prove(coset_index);
query_steps.push(FriQueryStep {
evals,
merkle_proof,
});
// 다음 레벨로 인덱스 이동
x_index >>= arity_bits;
}
```

각 단계에서:

- `evals` : 코셋의 모든 arity개 값들
- `merkle_proof` : 해당 코셋이 Merkle Tree에 포함되었음을 증명

Merkle Proof 생성 과정

Merkle Tree에서 특정 리프의 증명을 생성하는 과정:

```
Root

/ \

H₁  H₂  ← Sibling Hash (증명에 포함)

/ \ / \

H₃  H₄  H₅ H₆

/\ /\ /\ /\

L₀L₁  L₂ L₃  L₄ L₅ L₆L₇



L₂ 를 증명하려면:

- L₃  (형제)

- H₄  (삼촌)

- H₂  (대삼촌)
```

검증 과정:

1. L_2와 L_3를 해시 → H_3' 계산
2. H_3'와 H_4를 해시 → H_1' 계산
3. H_1'와 H_2를 해시 → Root' 계산
4. Root'가 커밋된 Root와 일치하는지 확인

FriQueryRound 구조

```
pub struct FriQueryRound<F: RichField + Extendable<D>, H: Hasher<F>, const D: u

pub initial_trees_proof: FriInitialTreeProof<F, H>,

pub steps: Vec<FriQueryStep<F, H, D>>,
```

```
}


pub struct FriQueryStep<F: RichField + Extendable<D>, H: Hasher<F>, const D: us
pub evals: Vec<F::Extension>,
pub merkle_proof: MerkleProof<F, H>,
}
```

구성 요소:

- `initial_trees_proof` : 원본 다항식들의 평가값과 증명
- `steps` : 각 FRI reduction 단계의 평가값과 증명

Query Phase 예시

arity = 2, 3번의 reduction을 가정:

```
x_index = 13 (이진수: 1101)


Initial: x_index = 13
- Initial trees에서 인덱스 13의 값과 증명 제공


Step 0: x_index = 13 >> 1 = 6
- Tree[0]에서 코셋 6의 값들 [v₁ ₂ , v₁ ₃ ] 제공
- 코셋 6의 Merkle Proof 제공


Step 1: x_index = 6 >> 1 = 3
```

```
- Tree[1]에서 코셋 3의 값들 [v6, v7] 제공
- 코셋 3의 Merkle Proof 제공

Step 2: x_index = 3 >> 1 = 1
- Tree[2]에서 코셋 1의 값들 [v2 , v3 ] 제공
- 코셋 1의 Merkle Proof 제공
```

Query Phase 요약

1. **쿼리 인덱스 선택**: Fiat-Shamir로 랜덤하게 여러 개 선택
2. **각 인덱스마다**:

- 원본 다항식들의 해당 위치 값과 Merkle Proof
- 각 FRI reduction 단계의 코셋 값들과 Merkle Proof

3. **병렬 처리**: 여러 쿼리를 동시에 처리하여 성능 향상

FRI Verification

Verification은 Verifier가 Prover의 증명을 검증하는 단계입니다.

verify_fri_proof 함수

```
pub fn verify_fri_proof<
F: RichField + Extendable<D>,
C: GenericConfig<D, F = F>,
```

```
const D: usize,
>(
instance: &FriInstanceInfo<F, D>,
openings: &FriOpenings<F, D>,
challenges: &FriChallenges<F, D>,
initial_merkle_caps: &[MerkleCap<F, C::Hasher>],
proof: &FriProof<F, C::Hasher, D>,
params: &FriParams,
) -> Result<()>
```

입력:

- `instance` : FRI 인스턴스 정보 (어떤 다항식들을 검증하는지)
- `openings` : 다항식들의 opening 값들
- `challenges` : 검증에 사용할 챌린지들 (alpha, betas, query indices)
- `initial_merkle_caps` : 원본 다항식들의 Merkle Cap
- `proof` : Prover가 제공한 FRI 증명
- `params` : FRI 파라미터

출력:

- 검증 성공 시 `Ok(())`
- 검증 실패 시 `Err(...)`

검증 단계

1단계: Proof of Work 검증

```
fri_verify_proof_of_work(challenges.fri_pow_response, &params.config)?;
```

```
pub(crate) fn  fri_verify_proof_of_work<F: RichField + Extendable<D>, const D: u
fri_pow_response: F,
config: &FriConfig,
) -> Result<()> {
ensure!(
fri_pow_response.to_canonical_u64().leading_zeros()
>= config.proof_of_work_bits + (64 - F::order().bits()) as u32,
"Invalid proof of work witness."
);
Ok(())
}
```

목적: Prover가 충분한 계산을 수행했는지 확인 (grinding)

검증:

- PoW 응답의 leading zeros 개수 확인
- 설정된 난이도 이상인지 검증

2단계: 각 쿼리 라운드 검증

```
let  precomputed_reduced_evals =
PrecomputedReducedOpenings::from_os_and_alpha(openings, challenges.fri_alpha);
```

```
for (&x_index, round_proof) in  challenges
.fri_query_indices
.iter()
.zip(&proof.query_round_proofs)
{
fri_verifier_query_round::<F, C, D>(
instance,
challenges,
&precomputed_reduced_evals,
initial_merkle_caps,
proof,
x_index,
n,
round_proof,
params,
)?;
}
```

과정:

- Opening 값들을 alpha로 미리 결합 (최적화)
- 각 쿼리 인덱스에 대해 검증 수행

fri_verifier_query_round 함수

```
fn  fri_verifier_query_round<
F: RichField + Extendable<D>,
C: GenericConfig<D, F = F>,
const D: usize,
>(
instance: &FriInstanceInfo<F, D>,
challenges: &FriChallenges<F, D>,
precomputed_reduced_evals: &PrecomputedReducedOpenings<F, D>,
initial_merkle_caps: &[MerkleCap<F, C::Hasher>],
proof: &FriProof<F, C::Hasher, D>,
mut  x_index: usize,
n: usize,
round_proof: &FriQueryRound<F, C::Hasher, D>,
params: &FriParams,
) -> Result<()>
```

단계 1: Initial Proof 검증

```
fri_verify_initial_proof::<F, C::Hasher>(
x_index,
&round_proof.initial_trees_proof,
initial_merkle_caps,
```

```
)?;


fn  fri_verify_initial_proof<F: RichField, H: Hasher<F>>(
x_index: usize,
proof: &FriInitialTreeProof<F, H>,
initial_merkle_caps: &[MerkleCap<F, H>],
) -> Result<()> {
for ((evals, merkle_proof), cap) in  proof.evals_proofs.iter().zip(initial_merkl
verify_merkle_proof_to_cap::<F, H>(
evals.clone(),
x_index,
cap,
merkle_proof
)?;
}
Ok(())
}
```

검증 내용:

- 각 원본 다항식의 평가값이 Merkle Tree에 포함되어 있는지 확인
- Merkle Proof를 사용하여 Cap까지 해시 경로 검증

단계 2: Subgroup 원소 계산

```
let log_n = log2_strict(n);
let mut subgroup_x = F::MULTIPLICATIVE_GROUP_GENERATOR
* F::primitive_root_of_unity(log_n).exp_u64(reverse_bits(x_index, log_n) as u64)
```

목적: x_index에 해당하는 실제 필드 원소 계산

계산 방법:

- `ω = primitive_root_of_unity(log_n)` : n차 단위근
- `subgroup_x = g * ω^(reverse_bits(x_index))` : 코셋의 원소
- `g = MULTIPLICATIVE_GROUP_GENERATOR` : 코셋 이동

단계 3: Initial Evaluation 결합

```
let mut old_eval = fri_combine_initial::<F, C, D>(
instance,
&round_proof.initial_trees_proof,
challenges.fri_alpha,
subgroup_x,
precomputed_reduced_evals,
params,
);
```

```
pub(crate) fn fri_combine_initial<
F: RichField + Extendable<D>,
```

```
C: GenericConfig<D, F = F>,
const D: usize,
>(
instance: &FriInstanceInfo<F, D>,
proof: &FriInitialTreeProof<F, C::Hasher>,
alpha: F::Extension,
subgroup_x: F,
precomputed_reduced_evals: &PrecomputedReducedOpenings<F, D>,
params: &FriParams,
) -> F::Extension {
let  subgroup_x = F::Extension::from_basefield(subgroup_x);
let  mut  alpha = ReducingFactor::new(alpha);
let  mut  sum = F::Extension::ZERO;


for (batch, reduced_openings) in  instance
.batches
.iter()
.zip(&precomputed_reduced_evals.reduced_openings_at_point)
{
let FriBatchInfo { point, polynomials } = batch;
let  evals = polynomials
.iter()
.map(|p| {
```

```
let poly_blinding = instance.oracles[p.oracle_index].blinding;
let salted = params.hiding && poly_blinding;
proof.unsalted_eval(p.oracle_index, p.polynomial_index, salted)
})
.map(F::Extension::from_basefield);
let reduced_evals = alpha.reduce(evals);
let numerator = reduced_evals - *reduced_openings;
let denominator = subgroup_x - *point;
sum = alpha.shift(sum);
sum += numerator / denominator;
}


sum
}
```

수학적 의미:

여러 다항식 $P_1, P_2, \ldots, P_n$을 하나로 결합:

$$\text{Combined}(x) = \Sigma_i \ \alpha^i \cdot (P_i(x) - P_i(z_i)) \ / \ (x - z_i)$$

여기서:

- α^i : Alpha의 거듭제곱 (랜덤 결합 계수)
- $P_i(x)$: i번째 다항식의 x에서의 평가값
- $P_i(z_i)$: i번째 다항식의 opening 값

- z_i : i번째 다항식의 opening 점

목적:

- 여러 다항식 검증을 하나의 FRI 검증으로 통합
- Alpha로 랜덤하게 결합하여 보안성 확보

단계 4: FRI Reduction Steps 검증

```
for (i, &arity_bits) in  params.reduction_arity_bits.iter().enumerate() {
let  arity = 1 << arity_bits;
let  evals = &round_proof.steps[i].evals;

// 코셋 인덱스 계산
let  coset_index = x_index >> arity_bits;
let  x_index_within_coset = x_index & (arity - 1);

// 일관성 확인
ensure!(evals[x_index_within_coset] == old_eval);

// 다음 레벨 평가값 계산
old_eval = compute_evaluation(
subgroup_x,
x_index_within_coset,
arity_bits,
evals,
```

```
challenges.fri_betas[i],
);

// Merkle Proof 검증
verify_merkle_proof_to_cap::<F, C::Hasher>(
flatten(evals),
coset_index,
&proof.commit_phase_merkle_caps[i],
&round_proof.steps[i].merkle_proof,
)?;

// 다음 레벨로 이동
subgroup_x = subgroup_x.exp_power_of_2(arity_bits);
x_index = coset_index;
}
```

각 단계에서:

1. **일관성 확인**:

- 이전 단계에서 계산한 값이 현재 단계의 코셋에 포함되어 있는지 확인
- `evals[x_index_within_coset] == old_eval`

2. **다음 평가값 계산**:

- `compute_evaluation` 함수로 folding 재현
- Beta 챌린지를 사용하여 다음 레벨의 값 계산

3. **Merkle Proof 검증**:

- 코셋의 값들이 커밋된 Merkle Tree에 포함되어 있는지 확인

4. **인덱스 업데이트**:

- `subgroup_x = subgroup_x^(2^arity_bits)` : 다음 레벨의 도메인 원소
- `x_index = coset_index` : 다음 레벨의 인덱스

compute_evaluation 함수 상세

```
pub(crate) fn compute_evaluation<F: Field + Extendable<D>, const D: usize>(
x: F,
x_index_within_coset: usize,
arity_bits: usize,
evals: &[F::Extension],
beta: F::Extension,
) -> F::Extension {
let arity = 1 << arity_bits;
debug_assert_eq!(evals.len(), arity);

let g = F::primitive_root_of_unity(arity_bits);

// 평가값 재정렬
let mut evals = evals.to_vec();
reverse_index_bits_in_place(&mut evals);
// 코셋 시작점 계산
```

```
let rev_x_index_within_coset = reverse_bits(x_index_within_coset, arity_bits);
let coset_start = x * g.exp_u64((arity - rev_x_index_within_coset) as u64);
// 보간 점들 생성
let points = g
.powers()
.map(|y| (coset_start * y).into())
.zip(evals)
.collect::<Vec<_>>();
// Barycentric 보간
let barycentric_weights = barycentric_weights(&points);
interpolate(&points, beta, &barycentric_weights)
}
```

수학적 과정:

1. **코셋 구성**: $\{x \cdot g^i \mid i = 0, \ldots, arity-1\}$
   - x: 현재 점
   - g: arity차 단위근
2. **보간**: 코셋의 점들 $\{(x \cdot g^i, P(x \cdot g^i))\}$ 로부터 다항식 P 복원
3. **평가**: 복원한 다항식을 beta에서 평가

결과: `P'(x^arity) = P(beta)`

- 이는 folding 과정을 역으로 검증하는 것입니다

Barycentric 보간

Lagrange 보간의 효율적인 형태:

$$P(z) = \Sigma_i\ w_i \cdot y_i\ /\ (z - x_i)\ /\ \Sigma_i\ w_i\ /\ (z - x_i)$$

여기서:

- w_i : Barycentric weight = 1 / $\prod_{j \neq i} (x_i - x_j)$`
- y_i : 점 x_i에서의 함수값
- z : 평가하려는 점 (여기서는 beta)

장점:

- O(n) 복잡도 (일반 Lagrange는 $O(n^2)$)
- 수치적으로 안정적

단계 5: Final Polynomial 검증

```
ensure!(
proof.final_poly.eval(subgroup_x.into()) == old_eval,
"Final polynomial evaluation is invalid."
);
```

최종 확인:

- 모든 reduction을 거친 후의 값이 최종 다항식과 일치하는지 확인
- 최종 다항식은 Prover가 직접 전송한 것입니다
- 이 값이 일치하면 전체 folding 과정이 올바름을 의미합니다

검증 과정 요약

```
1. PoW 검증
↓
2. 각 쿼리 인덱스마다:
├─ Initial trees Merkle Proof 검증
├─ Initial evaluation 결합 계산
├─ 각 FRI reduction 단계:
│  ├─ 이전 값과 일관성 확인
│  ├─ compute_evaluation으로 다음 값 계산
│  └─ Merkle Proof 검증
└─ Final polynomial과 일치 확인
↓
3. 모든 쿼리 통과 → 검증 성공
```

보안성 분석

Soundness:

- 잘못된 다항식을 사용하면 높은 확률로 검증 실패
- 쿼리 개수가 많을수록 soundness 향상
- 각 쿼리마다 독립적으로 검증

Completeness:

- 올바른 다항식이면 항상 검증 통과
- Folding 과정이 결정적이므로 재현 가능

Zero-Knowledge (선택적):

- Blinding을 사용하면 다항식 정보 숨김
- Salt를 추가하여 Merkle Tree 랜덤화

FRI 프로토콜 전체 흐름

Prover 관점

```
1. Setup
- 증명하려는 다항식 P(x) 준비
- LDE 수행: P(x)를 큰 도메인에서 평가

2. Commit Phase
For each reduction level:
- 평가값들을 arity개씩 묶어 Merkle Tree 생성
- Merkle Cap을 챌린저에 전달
- Beta 챌린지 받기
- 다항식을 Beta로 folding (차수 감소)
- 최종 다항식 전송

3. PoW Phase
- 충분한 leading zeros를 가진 witness 찾기

4. Query Phase
```

For each query index:

- Initial trees의 해당 위치 값과 Merkle Proof

- 각 reduction 단계의 코셋 값들과 Merkle Proof

Verifier 관점

1. Setup

- Prover의 커밋 (Merkle Caps) 받기

- 챌린지 생성 (alpha, betas, query indices)

2. PoW 검증

- Witness가 충분한 leading zeros를 가지는지 확인

3. Query Verification

For each query index:

- Initial trees Merkle Proof 검증

- Initial evaluation 결합

For each reduction level:

- 이전 값과 일관성 확인

- compute_evaluation으로 다음 값 계산

- Merkle Proof 검증

- Final polynomial과 일치 확인

```
4. 모든 쿼리 통과 → 증명 수락
```

복잡도 분석

Prover:

- Commit Phase: O(n log n) - FFT 연산
- Query Phase: O(k log n) - k개 쿼리, 각 log n 깊이

Verifier:

- Query Verification: O(k log n) - k개 쿼리, 각 log n 깊이
- 전체 다항식을 보지 않고도 검증 가능!

Proof Size:

- Merkle Caps: O(log n) per reduction level
- Query Proofs: O(k log n) - k개 쿼리
- Final Polynomial: O(1) - 충분히 작음

파라미터 선택:

- k (쿼리 개수): 보안 수준에 따라 결정 (보통 20-100)
- arity: 성능과 증명 크기 트레이드오프
- rate: 코드 레이트 (보통 1/2 또는 1/4)

관련 문서

- 03-proof-generation.md: FRI가 증명 생성에 사용되는 방법
- 04-proof-verification.md: FRI 검증이 전체 검증에 통합되는 방법
- 01-architecture.md: FRI 모듈의 전체 아키텍처

문서 네비게이션

- **이전**: 04. 증명 검증 프로세스 - 전체 검증 흐름
- **다음**: 06. Goldilocks Field - 필드 연산 구현
- **관련 문서**:
- 03. 증명 생성 프로세스 - FRI가 사용되는 증명 생성
- 04. 증명 검증 프로세스 - FRI 검증 통합
- 01. 아키텍처 개요 - FRI 모듈 구조